Universidad de La Habana · Facultad de Matemática y Computación
Ingeniería de Software

Clase práctica 4: Patrones de diseño orientado a objetos

Resolución de los ejercicios 7 y 11

Equipo CELL

Eduard González Prieto
Manuel Alejandro Rodríguez
Reynol Gómez Franco
Luis David Diaz Chiang
Cristian Miguel Castañeda Guzmán
Lázaro Javier Aragón García

Repositorio: https://github.com/ReynolGF/repo_IS_BD2_equipo_CELL

Índice

Entorno de ejecución **2**

Ejercicio 7. Sistema de Home Theater **2**

Inciso a): Patrón elegido, justificación y propuesta de solución 2
Inciso b): Implementación con 6 componentes y 3 actividades 4
Inciso c): Nueva actividad sin modificar la Fachada 5
Inciso d): Integración con Singleton y con Observer 7
Restricción: configurar la Fachada desde YAML 7

Ejercicio 11. Sistema de gestión de una cadena de suministro global **9**

Inciso a): Patrón elegido, justificación y propuesta de solución 9
Inciso b): Implementación con 3 modalidades, 4 familias y 2 variantes por producto 11
Inciso c): Prototype, Singleton y consistencia de las familias 12
Inciso d): Número de clases: herencia directa frente a Abstract Factory 14
Restricción: carga dinámica desde XML y validación de compatibilidad 14

Conclusiones **16**

Referencias **16**

Entorno de ejecución

.NET Framework 4.8 y el compilador de C# 5 de Windows, sin bibliotecas externas, en un equipo con 8 procesadores lógicos y .NET de 64 bits. La memoria se midió con `GC.GetTotalMemory(true)` y los tiempos con `Stopwatch`, por lo que son orientativos y varían entre equipos. Los costos de envío y los algoritmos son simulados. El código fuente completo está en la carpeta `codigo`, junto a la salida de cada ejecución; los programas de estos ejercicios son `Ej07_HomeTheater` y `Ej11_Logistica`.

Ejercicio 7. Sistema de Home Theater

Enunciado

Problema. Un sistema de home theater tiene múltiples componentes complejos (Amplificador, ReproductorDVD, Proyector, Pantalla, Luces). Los usuarios encuentran difícil operar todos estos componentes por separado para actividades comunes como «ver una película».

a) Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar Mediator o un simple método estático?

b) Implementa con 6 componentes y 3 actividades: VerPelícula, EscucharMúsica, JugarVideojuego.

c) Analiza: ¿Qué pasa si se quiere añadir una nueva actividad sin modificar el Facade? ¿Cómo lo resuelves con **Command** o **Strategy**?

d) Integra con **Singleton** para el Facade y con **Observer** para notificar cuando un componente cambia de estado.

Restricción. El Facade debe poder configurarse desde un archivo YAML para definir secuencias de actividades.

Inciso a): Patrón elegido, justificación y propuesta de solución

El patrón adecuado es **Facade (Fachada)**. En la conferencia se describe como una instancia que sirve de interfaz frontal y enmascara el código subyacente más complejo, mejorando la usabilidad al ofrecer una API única y a menudo simplificada; también puede dar interfaces de contexto específico a funcionalidades más genéricas. Gamma et al. (1994) lo resumen como una interfaz unificada para un conjunto de interfaces de un subsistema, de alto nivel, que hace el subsistema más fácil de usar.

Razonamiento

Las características del problema encajan una a una:

- Hay **varios componentes con interfaces propias** (Amplificador, ReproductorDVD, Proyector, Pantalla, Luces, y una consola para el tercer caso de uso) y el usuario los opera por separado.
- Las **actividades comunes son secuencias fijas de llamadas** sobre varios componentes (para ver una película: atenuar luces, bajar la pantalla, encender el

proyector, configurar el amplificador, reproducir). Ese conocimiento de «cómo se hace» es lo que la fachada concentra.

- Lo que se pide es **simplicidad para el cliente**, no cambiar el comportamiento de los componentes ni que se comuniquen entre sí.

¿Por qué no Mediator? La conferencia define Mediator como un objeto que encapsula cómo interactúan un conjunto de objetos y que promueve el bajo acoplamiento evitando que se refieran explícitamente unos a otros. Ese no es el problema aquí: los componentes del home theater no se hablan entre sí. El problema es la complejidad que ve el cliente. Además, la comunicación en un Mediator es en ambos sentidos (los colegas conocen al mediador y le notifican), mientras que en una Fachada es unidireccional: según la conferencia, las clases del subsistema deben «no tener conocimiento de la fachada y no mantener ninguna referencia a ella». Usar un Mediator obligaría a modificar los seis componentes.

¿Por qué no un simple método estático? Un método estático también simplificaría la llamada, pero es solo una agrupación procedimental: no tiene estado, no puede guardar las referencias a los componentes ni configurarse, no puede sustituirse por un doble en las pruebas, no implementa ninguna interfaz y no puede ser observado ni extendido con nuevas actividades. Los incisos c) y d) y la restricción necesitan precisamente un objeto con estado (registro de actividades, observadores, configuración desde un archivo).

Propuesta de solución: clases y relaciones

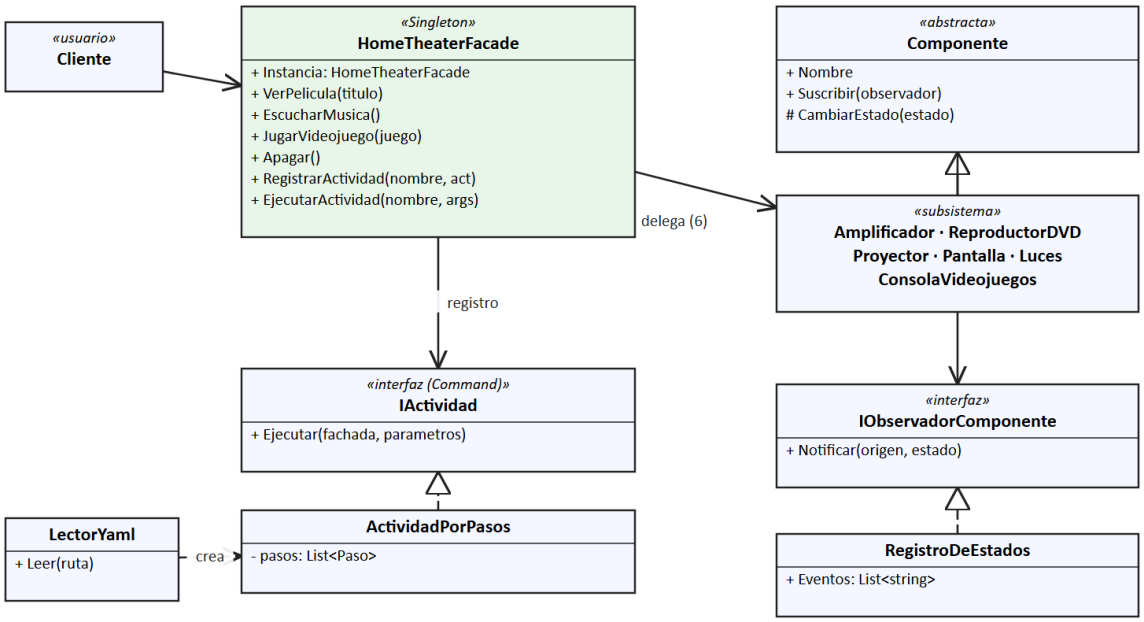


Figura 7.1. Fachada, subsistema, observadores y actividades extensibles. Elaboración propia.

Clase	Papel en el patrón	Relaciones
`HomeTheaterFacade`	Facade (y Singleton)	Conoce y delega en los seis componentes; guarda un registro de actividades
`Amplificador`, `ReproductorDVD`, `Proyector`, `Pantalla`, `Luces`, `ConsolaVideojuegos`	Clases del subsistema	Heredan de `Componente`; no conocen a la fachada
`Componente`	Subject del Observer	Mantiene la lista de `IObservadorComponente` y notifica cada cambio de estado
`RegistroDeEstados`	ConcreteObserver	Implementa `IObservadorComponente`; la fachada lo suscribe a todos los componentes
`IActividad`, `ActividadPorPasos`	Command	Una actividad configurable es una secuencia de pasos `componente.método`
`LectorYaml`	Apoyo	Convierte el archivo de configuración en objetos `IActividad`

Inciso b): Implementación con 6 componentes y 3 actividades

Los componentes avisan de cada cambio de estado a sus observadores. Este es el subsistema base y la fachada, con las tres actividades pedidas escritas como secuencias directas:

```
public interface IObservadorComponente
{
    void Notificar(Componente origen, string estado);
}

// Subsistema: los componentes no conocen a la Fachada; solo notifican a sus observadores.
public abstract class Componente
{
    private readonly List<IObservadorComponente> _observadores = new List<IObservadorComponente>();
    public string Nombre { get; private set; }
    protected Componente(string nombre) { Nombre = nombre; }

    public void Suscribir(IObservadorComponente o) { _observadores.Add(o); }
    public void Desuscribir(IObservadorComponente o) { _observadores.Remove(o); }

    protected void CambiarEstado(string estado)
    {
        Console.WriteLine("    " + Nombre + ": " + estado);
        foreach (var o in _observadores.ToArray()) o.Notificar(this, estado);
    }
}
```

Listado 1: Subsistema base: interfaz del observador y clase Componente

```
public sealed class HomeTheaterFacade
{
    // El inicializador estático lo ejecuta el CLR una sola vez y de forma segura entre hilos.
    private static readonly HomeTheaterFacade _instancia = new HomeTheaterFacade();
    public static HomeTheaterFacade Instancia { get { return _instancia; } }

    private readonly Amplificador _amp = new Amplificador();
    private readonly ReproductorDVD _dvd = new ReproductorDVD();
    private readonly Proyector _proy = new Proyector();
    private readonly Pantalla _pantalla = new Pantalla();
    private readonly Luces _luces = new Luces();
    private readonly ConsolaVideojuegos _consola = new ConsolaVideojuegos();
```

```
13
14      public readonly Dictionary<string, Componente> Componentes = new Dictionary<string, Componente>();
15      public readonly RegistroDeEstados Registro = new RegistroDeEstados();
16      private readonly Dictionary<string, IActividad> _actividades = new Dictionary<string, IActividad>(
    ↪ StringComparer.OrdinalIgnoreCase);
17
18      private HomeTheaterFacade()
19      {
20          foreach (Componente c in new Componente[] { _amp, _dvd, _proy, _pantalla, _luces, _consola })
21          {
22              Componentes[c.Nombre.ToLowerInvariant()] = c;
23              c.Suscribir(Registro);
24          }
25          // Alias cortos para el YAML
26          Componentes["amplificador"] = _amp; Componentes["dvd"] = _dvd; Componentes["proyector"] = _proy;
27          Componentes["pantalla"] = _pantalla; Componentes["luces"] = _luces; Componentes["consola"] =
    ↪ _consola;
28      }
29
30      // --- Las tres actividades de la fachada "clásica" ---
31      public void VerPelicula(string titulo)
32      {
33          Console.WriteLine("[Fachada] Preparando para ver \"" + titulo + "\"");
34          _luces.Atenuar(10);
35          _pantalla.Bajar();
36          _proy.Encender();
37          _proy.ModoPanoramico();
38          _amp.Encender();
39          _amp.Entrada("DVD");
40          _amp.SonidoEnvolvente();
41          _amp.Volumen(20);
42          _dvd.Encender();
43          _dvd.Reproducir(titulo);
44      }
45
46      // ... (continúa con RegistrarActividad, CargarConfiguracion y EjecutarActividad)
47  }
```

Listado 2: Fachada (Singleton): constructor y actividad VerPelicula

`EscucharMusica` y `JugarVideojuego` siguen el mismo esquema con sus propias secuencias; el código completo está en `Ej07_HomeTheater/Program.cs`. Salida de la ejecución para la primera actividad:

```
Singleton: misma instancia

[Fachada] Preparando para ver "Matrix"
    Luces: atenuadas al 10%
    Pantalla: bajada
    Proyector: encendido
    Proyector: modo panorámico
    Amplificador: encendido
    Amplificador: entrada DVD
    Amplificador: sonido envolvente
    Amplificador: volumen 20
    ReproductorDVD: encendido
    ReproductorDVD: reproduciendo "Matrix"
```

Inciso c): Nueva actividad sin modificar la Fachada

Si cada actividad fuera un método de la fachada, añadir una exigiría modificar y recompilar la clase. La solución es sacar la actividad de la fachada y tratarla como un objeto. La conferencia describe **Command** como el patrón que encapsula en un objeto toda la información necesaria para realizar una acción en un momento posterior; una actividad del home theater es exactamente eso: una secuencia de llamadas, con parámetros, que se puede registrar con un nombre y ejecutar cuando se pida. Por eso se escogió Command y no Strategy: Strategy sirve para elegir entre algoritmos intercambiables que hacen *lo mismo* de distinta forma, mientras

que aquí las actividades hacen cosas distintas y lo que se necesita es poder almacenarlas, nombrarlas y parametrizarlas (y, en el futuro, deshacerlas).

```
public interface IActividad
{
    void Ejecutar(HomeTheaterFacade fachada, string[] parametros);
}

public class Paso
{
    public string Componente, Metodo;
    public string[] Argumentos;
}

// Actividad descrita como una secuencia de pasos (viene del YAML).
public class ActividadPorPasos : IActividad
{
    private readonly List<Paso> _pasos;
    public ActividadPorPasos(List<Paso> pasos) { _pasos = pasos; }

    public void Ejecutar(HomeTheaterFacade fachada, string[] parametros)
    {
        foreach (var p in _pasos)
        {
            Componente c;
            if (!fachada.Componentes.TryGetValue(p.Componente.ToLowerInvariant(), out c))
                throw new InvalidOperationException("Componente desconocido: " + p.Componente);
            MethodInfo m = c.GetType().GetMethod(p.Metodo, BindingFlags.Public | BindingFlags.Instance |
↪ BindingFlags.IgnoreCase);
            if (m == null)
                throw new InvalidOperationException("Método desconocido: " + p.Componente + "." + p.Metodo)
↪ ;
            ParameterInfo[] ps = m.GetParameters();
            if (ps.Length != p.Argumentos.Length)
                throw new InvalidOperationException("Número de argumentos incorrecto en " + p.Componente +
↪ "." + p.Metodo);
            object[] args = new object[ps.Length];
            for (int i = 0; i < ps.Length; i++)
            {
                string a = p.Argumentos[i];
                if (a.StartsWith("$")) a = parametros[int.Parse(a.Substring(1))];   // $0 = primer
↪ parámetro de la actividad
                args[i] = Convert.ChangeType(a, ps[i].ParameterType);
            }
            m.Invoke(c, args);
        }
    }
}
```

Listado 3: Command: IActividad, Paso y ActividadPorPasos

```
    // Miembros que se añaden a HomeTheaterFacade:
    // --- Extensión sin modificar la clase: actividades registradas desde fuera ---
    public void RegistrarActividad(string nombre, IActividad actividad) { _actividades[nombre] = actividad;
↪ }

    public void CargarConfiguracion(string rutaYaml)
    {
        foreach (var kv in LectorYaml.Leer(rutaYaml)) RegistrarActividad(kv.Key, kv.Value);
    }

    public void EjecutarActividad(string nombre, params string[] parametros)
    {
        IActividad a;
        if (!_actividades.TryGetValue(nombre, out a))
            throw new KeyNotFoundException("Actividad no registrada: " + nombre);
        Console.WriteLine("[Fachada] Ejecutando actividad configurada \"" + nombre + "\"");
        a.Ejecutar(this, parametros);
    }
}
```

Listado 4: Registro y ejecución de actividades en la fachada

Con esto la fachada solo conserva un diccionario nombre → `IActividad`. Una actividad nueva es una clase nueva que implementa `IActividad`, o una entrada nueva en el archivo YAML, y se registra con `RegistrarActividad`. La clase `HomeTheaterFacade` no cambia. La ejecución lo comprueba: `NocheDeCine` no existe en el código y se ejecuta desde el YAML (véase la restricción).

Cuidado

Limitación. Esto elimina la modificación de la fachada para nuevas *actividades*. Un nuevo *tipo de componente* (por ejemplo, un altavoz adicional) sí exige registrarlo en el constructor de la fachada, porque es ella quien los crea.

ActividadPorPasos es el Command concreto que encapsula los pasos; la fachada actúa como el Invoker.

Inciso d): Integración con Singleton y con Observer

Singleton. La conferencia lo define como el patrón que restringe la instanciación de una clase a una única instancia y es útil cuando se necesita exactamente una para coordinar las acciones de todo el sistema. Hay una sola sala con un solo conjunto de componentes físicos, así que dos fachadas competirían por los mismos dispositivos. Se implementó con un campo `static readonly` inicializado en el constructor privado: el CLR ejecuta el inicializador estático una sola vez y de forma segura entre hilos, y la ejecución confirma que `Instancia` devuelve siempre el mismo objeto. El precio del Singleton es el estado global, que dificulta las pruebas; por eso conviene que los clientes reciban la fachada por parámetro y no la busquen en cualquier parte del código.

Observer. La conferencia lo describe como un sujeto que mantiene una lista de observadores y los notifica automáticamente de cualquier cambio de estado. Cada `Componente` es un sujeto y `CambiarEstado` notifica a sus observadores. La fachada suscribe un `RegistroDeEstados` a los seis componentes; en la ejecución, que incluye las tres actividades, sus apagados y la actividad del YAML, el observador recibió **52 eventos** y el último fue «ReproductorDVD → reproduciendo “Casablanca”». Los componentes siguen sin conocer a la fachada: solo conocen la interfaz `IObservadorComponente`. Otros observadores (una pantalla de estado, un registro en archivo) se añaden sin tocar el subsistema.

Restricción: configurar la Fachada desde YAML

Las secuencias de actividades se leen de un archivo YAML. Cada paso tiene la forma `componente.método argumentos`, y `$0` representa el primer parámetro con que se invoca la actividad:

```yaml
# Actividades del home theater. Cada paso es: componente.metodo argumentos
# $0 es el primer parámetro con el que se invoca la actividad.
actividades:
  NocheDeCine:
    - luces.Atenuar 5
    - pantalla.Bajar
    - proyector.Encender
    - amplificador.Encender
    - amplificador.Entrada "DVD"
    - amplificador.Volumen 25
    - dvd.Encender
    - dvd.Reproducir $0
  ModoLectura:
    - luces.Atenuar 80
    - amplificador.Apagar
```

Listado 5: Archivo homeTheater.yaml

`LectorYaml` convierte cada actividad en una `ActividadPorPasos`, y esta localiza el

componente por nombre y el método por reflexión, convierte los argumentos al tipo del parámetro y lo invoca. Si el componente, el método o el número de argumentos no existen, falla con un mensaje explícito en lugar de ignorar el paso. Resultado de la ejecución con el archivo anterior:

```
--- Actividades definidas en homeTheater.yaml (sin recompilar) ---
[Fachada] Ejecutando actividad configurada "NocheDeCine"
    Luces: atenuadas al 5%
    Pantalla: bajada
    Proyector: encendido
    Amplificador: encendido
    Amplificador: entrada DVD
    Amplificador: volumen 25
    ReproductorDVD: encendido
    ReproductorDVD: reproduciendo "Casablanca"

Error esperado: Actividad no registrada: Karaoke

Eventos recibidos por el observador: 52
Último evento: ReproductorDVD -> reproduciendo "Casablanca"
```

Cuidado

Limitación. El lector entiende solo el subconjunto de YAML que se muestra arriba (una lista de pasos por actividad). Para YAML completo (anclas, estructuras anidadas, tipos) haría falta una biblioteca de análisis; no se usó ninguna para no añadir dependencias al ejercicio.

Ejercicio 11. Sistema de gestión de una cadena de suministro global

Enunciado

Problema. Una empresa multinacional maneja una cadena de suministro con múltiples proveedores, centros de distribución y métodos de transporte (marítimo, aéreo, terrestre). Cada tipo de transporte tiene sus propias reglas de empaque, documentación y costos. La empresa necesita que los objetos de una misma familia logística (ej. «Envío Marítimo» con su «Documentación Marítima» y su «Seguro Marítimo») sean consistentes entre sí, y que no se mezclen elementos de diferentes modalidades en un mismo envío. Además, en el futuro se planea añadir nuevas modalidades como «transporte ferroviario» o «drones de entrega».

a) Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar Factory Method simple o Builder?

b) Implementa con al menos 3 modalidades (Marítimo, Aéreo, Terrestre) y 4 familias de productos (Empaque, Documentación, Seguro, Rastreo). Cada producto debe tener al menos 2 variantes por modalidad.

c) Integra con **Prototype** para clonar envíos predefinidos (ej. «Envío estándar marítimo») y con **Singleton** para el catálogo de fábricas logísticas. ¿Cómo afecta el clonado a la consistencia de las familias?

d) Calcula el número de clases si se usara herencia directa vs. Abstract Factory. ¿Cómo escala con N modalidades y M productos? ¿Qué pasa si N=10 y M=8? ¿Es viable la herencia directa?

Restricción. El sistema debe cargar las fábricas dinámicamente desde un archivo de configuración (XML/JSON) sin recompilar. Además, cada fábrica debe validar que los productos que crea sean compatibles entre sí (ej. un envío marítimo no puede usar seguro aéreo).

Inciso a): Patrón elegido, justificación y propuesta de solución

El patrón adecuado es **Abstract Factory (Fábrica Abstracta)**. La conferencia lo presenta como una interfaz para crear **familias de objetos** sin depender de sus clases concretas; sus notas añaden que agrupa varias fábricas individuales con funcionalidades comunes, y que el cliente no necesita saber qué instancias concretas recibe porque las usa de forma genérica.

Razonamiento

El enunciado pide exactamente eso:

- **Familias de productos relacionados**: Empaque, Documentación, Seguro y Rastreo de una misma modalidad.
- **Consistencia dentro de la familia y prohibición de mezclar**: un envío marítimo no puede llevar un seguro aéreo.
- **Familias nuevas en el futuro** (ferroviario, drones): deben poder añadirse sin tocar el código existente.

¿Por qué no Factory Method simple? Factory Method (conferencia) define una interfaz para crear *un* objeto y deja que las subclases decidan la clase concreta. Para los cuatro productos habría cuatro creadores independientes, y nada garantizaría que el cliente elija el creador marítimo para los cuatro: la consistencia dependería de la disciplina del cliente. Abstract Factory reúne los cuatro métodos de creación en un único objeto por modalidad; la familia se elige una sola vez, al elegir la fábrica, y todos los productos salen coherentes.

¿Por qué no Builder? Builder (conferencia) separa la construcción de un objeto complejo de su representación para que el mismo proceso cree distintas representaciones. Es adecuado para ensamblar paso a paso *un* objeto, pero no obliga a que las partes pertenezcan a la misma familia: un constructor al que se le pasan partes de distintas modalidades las aceptaría. Los dos patrones pueden combinarse (un constructor de envíos que obtiene sus partes de una fábrica abstracta), pero Builder por sí solo no resuelve la consistencia.

Propuesta de solución: clases y relaciones

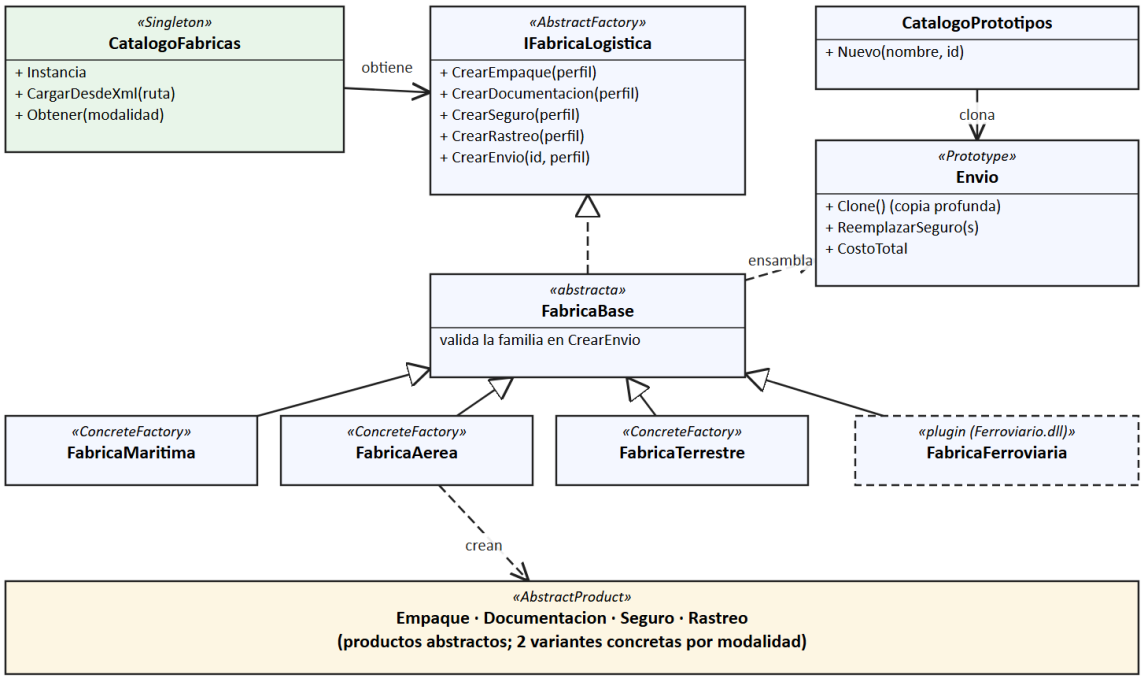


Figura 11.1. Abstract Factory con Prototype y Singleton; la modalidad ferroviaria entra como plugin. Elaboración propia.

Clase	Papel	Descripción
`IFabricaLogistica`	AbstractFactory	Un método de creación por producto, más `CrearEnvio`; el parámetro `Perfil` (General o Especial) elige entre las 2 variantes
`FabricaMaritima`, `FabricaAerea`, `FabricaTerrestre`	ConcreteFactory	Cada una crea los 4 productos de su modalidad
`Empaque`, `Documentacion`, `Seguro`, `Rastreo`	AbstractProduct	Productos abstractos; cada uno conoce su modalidad
24 clases como `ContenedorSeco` o `GuiaAerea`	ConcreteProduct	3 modalidades × 4 productos × 2 variantes
`Envio`	Prototype	Producto compuesto que valida su propia coherencia y se clona
`CatalogoFabricas`	Singleton	Registro único de fábricas, cargado desde XML
`CatalogoPrototipos`	Registro de prototipos	Envíos predefinidos que se clonan

Inciso b): Implementación con 3 modalidades, 4 familias y 2 variantes por producto

Cada modalidad tiene 4 productos con 2 variantes cada uno. La fábrica marítima, como ejemplo, y la base común que valida la familia:

```
public interface IFabricaLogistica
{
    string Modalidad { get; }
    Empaque CrearEmpaque(Perfil p);
    Documentacion CrearDocumentacion(Perfil p);
    Seguro CrearSeguro(Perfil p);
    Rastreo CrearRastreo(Perfil p);
    Envio CrearEnvio(string id, Perfil p);
}

// Base común: crea el envío completo y valida la coherencia de la familia.
public abstract class FabricaBase : IFabricaLogistica
{
    public abstract string Modalidad { get; }
    public abstract Empaque CrearEmpaque(Perfil p);
    public abstract Documentacion CrearDocumentacion(Perfil p);
    public abstract Seguro CrearSeguro(Perfil p);
    public abstract Rastreo CrearRastreo(Perfil p);

    public Envio CrearEnvio(string id, Perfil p)
    {
        Empaque e = CrearEmpaque(p); Documentacion d = CrearDocumentacion(p); Seguro s = CrearSeguro(p);
↪ Rastreo r = CrearRastreo(p);
        foreach (Producto prod in new Producto[] { e, d, s, r })
            if (prod.Modalidad != Modalidad)
                throw new IncompatibilidadLogisticaException("La fábrica " + Modalidad + " produjo un
↪ producto de modalidad " + prod.Modalidad);
        return new Envio(id, Modalidad, e, d, s, r);
    }
}
```

Listado 6: Fábrica abstracta y base común que valida la familia

```
public class ContenedorSeco : Empaque { public ContenedorSeco() : base("Maritimo", "Contenedor seco 40'",
↪ 1200m) { } }
public class ContenedorRefrigerado : Empaque { public ContenedorRefrigerado() : base("Maritimo", "
```

```
    ↪ Contenedor refrigerado 40'", 2600m) { } }
3 public class ConocimientoEmbarque : Documentacion { public ConocimientoEmbarque() : base("Maritimo", "
    ↪ Conocimiento de embarque (B/L)", 150m) { } }
4 public class ConocimientoEmbarqueSanitario : Documentacion { public ConocimientoEmbarqueSanitario() : base(
    ↪ "Maritimo", "B/L con certificado sanitario", 320m) { } }
5 public class SeguroMaritimoBasico : Seguro { public SeguroMaritimoBasico() : base("Maritimo", "Seguro
    ↪ marítimo básico", 400m) { } }
6 public class SeguroMaritimoAmplio : Seguro { public SeguroMaritimoAmplio() : base("Maritimo", "Seguro
    ↪ marítimo todo riesgo", 950m) { } }
7 public class RastreoMaritimoAIS : Rastreo { public RastreoMaritimoAIS() : base("Maritimo", "Rastreo AIS del
    ↪  buque", 60m) { } }
8 public class RastreoMaritimoContenedor : Rastreo { public RastreoMaritimoContenedor() : base("Maritimo", "
    ↪ Rastreo GPS del contenedor", 180m) { } }
9
10 public class FabricaMaritima : FabricaBase
11 {
12     public override string Modalidad { get { return "Maritimo"; } }
13     public override Empaque CrearEmpaque(Perfil p) { return p == Perfil.General ? (Empaque)new
    ↪ ContenedorSeco() : new ContenedorRefrigerado(); }
14     public override Documentacion CrearDocumentacion(Perfil p) { return p == Perfil.General ? (
    ↪ Documentacion)new ConocimientoEmbarque() : new ConocimientoEmbarqueSanitario(); }
15     public override Seguro CrearSeguro(Perfil p) { return p == Perfil.General ? (Seguro)new
    ↪ SeguroMaritimoBasico() : new SeguroMaritimoAmplio(); }
16     public override Rastreo CrearRastreo(Perfil p) { return p == Perfil.General ? (Rastreo)new
    ↪ RastreoMaritimoAIS() : new RastreoMaritimoContenedor(); }
17 }
```

Listado 7: Productos concretos marítimos y FabricaMaritima

Salida de la ejecución: un envío por modalidad y por perfil (los costos son simulados).

```
== Un envío por modalidad y perfil (costos simulados) ==
MAR-0 [Maritimo] Contenedor seco 40' | Conocimiento de embarque (B/L) | Seguro marítimo básico | Rastreo AIS del
↪  buque => 1810.00
MAR-1 [Maritimo] Contenedor refrigerado 40' | B/L con certificado sanitario | Seguro marítimo todo riesgo |
↪ Rastreo GPS del contenedor => 4050.00
AER-0 [Aereo] Paleta aérea ULD | Guía aérea (AWB) | Seguro aéreo básico | Rastreo por estados de vuelo =>
↪ 1330.00
AER-1 [Aereo] Caja térmica aérea | AWB con declaración de mercancía peligrosa | Seguro aéreo con valor declarado
↪  | Rastreo en tiempo real => 2850.00
TER-0 [Terrestre] Cajas sobre paleta | Carta de porte | Seguro terrestre básico | Rastreo GPS del camión =>
↪ 490.00
TER-1 [Terrestre] Furgón refrigerado | Carta de porte internacional | Seguro terrestre ampliado | Rastreo GPS y
↪ temperatura => 1250.00
```

Inciso c): Prototype, Singleton y consistencia de las familias

Prototype. La conferencia lo describe como el patrón en que el tipo de la instancia creada lo determina una instancia prototipo, que se clona para producir objetos nuevos, evitando el costo de crearlos de la forma estándar. `Envio` implementa `ICloneable`, y `CatalogoPrototipos` guarda envíos predefinidos como «Envío estándar marítimo»; `Nuevo(nombre, id)` devuelve un clon con identificador propio. **Singleton.** `CatalogoFabricas` tiene una única instancia con las fábricas disponibles, que se carga una vez desde la configuración.

```
1 public class Envio : ICloneable
2 {
3     public string Id { get; private set; }
4     public string Modalidad { get; private set; }
5     public Empaque Empaque { get; private set; }
6     public Documentacion Documentacion { get; private set; }
7     public Seguro Seguro { get; private set; }
8     public Rastreo Rastreo { get; private set; }
9
10    public Envio(string id, string modalidad, Empaque e, Documentacion d, Seguro s, Rastreo r)
11    {
12        Id = id; Modalidad = modalidad;
13        Validar(e); Validar(d); Validar(s); Validar(r);
14        Empaque = e; Documentacion = d; Seguro = s; Rastreo = r;
15    }
```

```
16
17      private void Validar(Producto p)
18      {
19          if (p.Modalidad != Modalidad)
20              throw new IncompatibilidadLogisticaException("Un envío " + Modalidad + " no admite \"" + p.
    ↪ Descripcion + "\" (modalidad " + p.Modalidad + ")");
21      }
22
23      // Los reemplazos pasan por la misma validación que la construcción.
24      public void ReemplazarSeguro(Seguro s) { Validar(s); Seguro = s; }
25      public void ReemplazarEmpaque(Empaque e) { Validar(e); Empaque = e; }
26
27      public decimal CostoTotal { get { return Empaque.Costo + Documentacion.Costo + Seguro.Costo + Rastreo.
    ↪ Costo; } }
28
29      // Copia profunda: el clon no comparte productos con el original.
30      public object Clone()
31      {
32          return new Envio(Id, Modalidad, (Empaque)Empaque.Clone(), (Documentacion)Documentacion.Clone(), (
    ↪ Seguro)Seguro.Clone(), (Rastreo)Rastreo.Clone());
33      }
34      public Envio ClonarConId(string id)
35      {
36          Envio c = (Envio)Clone(); c.Id = id; return c;
37      }
38
39      public override string ToString()
40      {
41          return Id + " [" + Modalidad + "] " + Empaque.Descripcion + " | " + Documentacion.Descripcion + " |
    ↪  " + Seguro.Descripcion + " | " + Rastreo.Descripcion + " => " + CostoTotal.ToString("F2");
42      }
43  }
```

Listado 8: Envio como Prototype: validación y copia profunda

¿Cómo afecta el clonado a la consistencia de las familias? Un clon de un envío válido nace con una familia válida, pero la consistencia se mantiene solo si se cumplen tres condiciones:

Razonamiento

- **La copia es profunda.** El clon no comparte productos con el original; si fuera superficial, modificar un producto del clon cambiaría el prototipo y, con él, todos los envíos futuros. La ejecución lo verifica: al ampliar el seguro del clon `E-1002`, el original `E-1001` conserva su seguro básico y `ReferenceEquals` devuelve falso.
- **Las modificaciones pasan por la misma validación.** `ReemplazarSeguro` y `ReemplazarEmpaque` comprueban la modalidad. Sin esa comprobación, un clon permitiría la mezcla que la fábrica impide.
- **Los productos son inmutables.** Así `MemberwiseClone` es una copia segura de cada producto; si un producto tuviera referencias a objetos mutables, haría falta una copia profunda también ahí.

Cuidado

Un riesgo adicional es que un prototipo es una instantánea: si las reglas de una fábrica cambian, los prototipos ya registrados conservan los productos antiguos, y habría que regenerarlos.

Nota. Las clases de producto implementan ICloneable mediante MemberwiseClone(), que es seguro porque son inmutables.

Inciso d): Número de clases: herencia directa frente a Abstract Factory

Para poder contar hay que fijar qué es «herencia directa». Se entiende como el diseño sin fábricas en el que cada combinación posible de variantes de una modalidad es una subclase concreta de `Envio` que fija sus cuatro productos (por ejemplo, `EnvioMaritimoContenedorSecoConSeguroAmplio…`). Se llama **N** al número de modalidades, **M** al de familias de productos y **v** al de variantes por producto y modalidad. Los productos concretos ($N \cdot M \cdot v$ clases) hacen falta en ambos diseños, así que la diferencia está en las clases estructurales:

Diseño	Clases estructurales y totales	Crecimiento
Herencia directa	Estructurales: $N \cdot v^M$ subclases de envío. Totales: $N \cdot v^M + N \cdot M \cdot v$.	Exponencial en M
Abstract Factory	Estructurales: $N + M + 1$. Totales: $N + M + 1 + N \cdot M \cdot v$.	Aditivo

Caso	Productos concretos (ambos)	Herencia directa (total)	Abstract Factory (total)
$N = 3$, $M = 4$, $v = 2$ (la implementación)	24	$48 + 24 = 72$	$8 + 24 = 32$
$N = 10$, $M = 8$, $v = 2$	160	$2.560 + 160 = 2.720$	$19 + 160 = 179$
$N = 10$, $M = 8$, $v = 1$	80	$10 + 80 = 90$	$19 + 80 = 99$

El programa confirma el primer caso: la reflexión sobre el ensamblado cuenta **24 clases concretas de producto y 3 fábricas concretas** (más `FabricaBase` como apoyo común).

¿Cómo escala? Con N modalidades y M productos, la herencia directa crece como $N \cdot v^M$: con $N = 10$, $M = 8$ y dos variantes por producto hay **2.560** clases, y cada nueva familia de productos duplica el número de clases de cada modalidad. **No es viable.** Abstract Factory crece de forma aditiva: añadir una modalidad cuesta 1 fábrica y $M \cdot v$ productos y no modifica ninguna clase existente (en la prueba, el plugin ferroviario son 8 productos y 1 fábrica); añadir una variante a un producto no cambia la estructura, porque la variante se elige con el parámetro.

Cuidado

Dos matices de honestidad. Con **una sola variante** por producto ($v = 1$) la herencia directa tiene menos clases totales (90 frente a 99), así que el argumento a favor de Abstract Factory en ese caso es la consistencia garantizada y la extensión sin modificar, no el conteo. Y la debilidad conocida de Abstract Factory es la inversa: **añadir una nueva familia de productos** (por ejemplo «Aduana») obliga a añadir un método a la interfaz y a modificar las N fábricas.

Restricción: carga dinámica desde XML y validación de compatibilidad

Carga dinámica. Las fábricas se declaran en un XML y se instancian por reflexión; para una modalidad externa se indica además el ensamblado. Añadir la modalidad ferroviaria **no requirió recompilar la aplicación**: `Ferroviario.dll` se compiló por separado, con su `FabricaFerroviaria`, y bastó una línea más en el XML:

```xml
<?xml version="1.0" encoding="utf-8"?>
<fabricas>
  <fabrica modalidad="Maritimo" tipo="Logistica.FabricaMaritima" />
  <fabrica modalidad="Aereo" tipo="Logistica.FabricaAerea" />
  <fabrica modalidad="Terrestre" tipo="Logistica.FabricaTerrestre" />
  <fabrica modalidad="Ferroviario" tipo="Ferroviario.FabricaFerroviaria" ensamblado="Ferroviario.dll" />
</fabricas>
```

Listado 9: Archivo fabricas_con_ferroviario.xml

```
public void CargarDesdeXml(string ruta)
{
    _fabricas.Clear();
    foreach (XElement x in XDocument.Load(ruta).Root.Elements("fabrica"))
    {
        string modalidad = (string)x.Attribute("modalidad"), tipo = (string)x.Attribute("tipo"), dll = (
↪ string)x.Attribute("ensamblado");
        Assembly ensamblado = dll == null ? Assembly.GetExecutingAssembly() : Assembly.LoadFrom(System.IO.
↪ Path.Combine(AppDomain.CurrentDomain.BaseDirectory, dll));
        Type t = ensamblado.GetType(tipo, true);
        var f = Activator.CreateInstance(t) as IFabricaLogistica;
        if (f == null) throw new InvalidOperationException(tipo + " no implementa IFabricaLogistica");
        if (!string.Equals(f.Modalidad, modalidad, StringComparison.OrdinalIgnoreCase))
            throw new InvalidOperationException("La configuración declara " + modalidad + " pero " + tipo +
↪  " produce " + f.Modalidad);
        _fabricas[modalidad] = f;
    }
}
```

Listado 10: Carga de fábricas por reflexión desde el XML

La misma aplicación, sin recompilar, se ejecutó con los dos archivos. Con `fabricas.xml` cargó tres modalidades; con `fabricas_con_ferroviario.xml` cargó cuatro y generó además los envíos `FER-0` y `FER-1`:

```
Configuración: fabricas_con_ferroviario.xml
Modalidades cargadas: Maritimo, Aereo, Terrestre, Ferroviario
...
FER-0 [Ferroviario] Vagón cerrado | Carta de porte ferroviaria | Seguro ferroviario básico | Rastreo por
↪ estaciones => 785.00
FER-1 [Ferroviario] Vagón frigorífico | Carta de porte ferroviaria internacional | Seguro ferroviario ampliado |
↪  Rastreo GPS del vagón => 1895.00
```

Si la configuración declara una modalidad que la clase no produce, o una clase que no implementa `IFabricaLogistica`, la carga falla con un mensaje explícito.

Validación de compatibilidad. Se aplica en dos puntos: la fábrica comprueba, en `CrearEnvio`, que los cuatro productos que creó pertenezcan a su modalidad, y el propio `Envio` valida cada producto al construirse y al reemplazarlo. La ejecución prueba los dos caminos de rechazo:

```
== Validación de compatibilidad ==
Rechazado al reemplazar: Un envío Maritimo no admite "Seguro aéreo básico" (modalidad Aereo)
Rechazado al construir: Un envío Maritimo no admite "Guía aérea (AWB)" (modalidad Aereo)
```

Conclusiones

Conclusión

Cada ejercicio trabajó un patrón distinto con el mismo criterio: identificarlo por el problema que resuelve y no por el nombre, comprobarlo con código ejecutable y señalar su costo. Facade simplificó el uso de un subsistema sin tocar sus componentes, y Abstract Factory garantizó familias consistentes con un número de clases que crece de forma aditiva.

Dos ideas se repitieron. La primera es que los patrones se combinan: Facade se integró con Command, Singleton y Observer, y Abstract Factory con Prototype y Singleton, y cada uno aportó una responsabilidad distinta (crear, extender o notificar). La segunda es que ningún patrón es gratuito: Singleton añade estado global y Abstract Factory vuelve costosa la incorporación de nuevas familias de producto.

Referencias

Asignatura Ingeniería de Software. (s. f.). *Marcos de Trabajo y Patrones de Diseño* [Presentación con notas del orador]. Universidad de La Habana.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Bibliografía citada en la conferencia anterior; se cita por lo que la conferencia recoge de ella, sin referencia de página.)